

Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 25.М41-мм

Улучшение качества кода проекта Desbordante

ШЛЁНСКИХ Алексей Анатольевич

Отчёт по учебной практике
в форме «Производственное задание»

Научный руководитель:
ассистент кафедры ИАС, Г. А. Чернышев.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	4
2. Текущее состояние системы	5
2.1. Запуск алгоритма	6
2.2. Система непрерывной интеграции	8
2.3. Зависимости	9
2.4. Сборка	11
2.5. Поиск функциональных зависимостей	14
2.6. Детали изменённых алгоритмов	15
2.7. Зависимости включения	16
2.8. Случайные ошибки тестирования	17
2.9. Устаревший код	17
2.10. Недостающие возможности	21
3. Проведённая работа	22
3.1. Процесс сборки	22
3.2. Автоматические проверки кода	24
3.3. Неиспользуемая функциональность	26
3.4. Повторный обзор кода	28
3.5. Добавление недостающих возможностей	29
4. Анализ	32
4.1. Проверка эффектов скрытых символов	32
4.2. Уменьшение количества кода	33
4.3. Система непрерывной интеграции	34
4.4. Возможности Desbordante	36
Заключение	39
Список литературы	40

Введение

Проект Desbordante является библиотекой для профилирования данных, работающей с различными типами закономерностей, написанной на C++. Библиотека состоит из алгоритмов для поиска соблюдающихся и проверки соблюдения условий некоторых классов.

Изначально проект предназначался для измерения производительности таких алгоритмов и разрабатывался небольшой командой для написания научных статей наподобие [4], однако с течением времени его позиционирование поменялось: были добавлены связка с Python, тестирование с помощью GitHub Actions, была переработана система сборки. В разработке проекта участвовало много людей, основная часть которых — студенты, у которых были различные внешние ограничения (например, необходимость вовремя написать отчёт по практике).

Из-за этих обстоятельств в проекте накапливался технический долг, который не устранялся своевременно, поскольку многие участники делали свою часть и покидали проект.

Между стадиями инструмента для экспериментов и библиотеки для Python, проект также был представлен в виде веб-приложения, имевшего тесно интегрированный с основным кодом клиент, что тоже отразилось на коде.

Из-за отсутствия документации процесс добавления алгоритма в проект сводится к написанию по подобию уже написанного. Из-за обозначенных выше особенностей проекта это доставляет особенно много проблем, ведь некоторый код относится к более неиспользуемым функциям или написан неоптимально, что ведёт к распространению ошибок. Также устаревший код затрудняет адаптацию к новым реалиям проекта и обучение новых участников команды, в итоге создавая острые углы для пользователей.

В данном отчёте приведена проведённая автором работа по уменьшению влияния данных проблем и реализации некоторой недостающей функциональности.

1. Постановка задачи

Целью работы является улучшение качества кода проекта Desbor-dante и уменьшение количества сбивающих с толку элементов. Для её выполнения были поставлены следующие задачи:

1. Выполнить повторный просмотр части написанного кода.
2. Устранить обнаруженные программные ошибки.
3. Упростить существующий код.
4. Добавить недостающие возможности.
5. Проанализировать результаты для дальнейшего улучшения про-екта.

2. Текущее состояние системы

Проект Desbordante на самом высоком уровне представляет набор алгоритмов для поиска соблюдающихся и проверки условий определённых классов. Внутренняя работа алгоритмов и конкретные искомые условия могут регулироваться с помощью параметров.

Алгоритмы поиска часто крайне ресурсоёмки по времени и памяти. Даже для одного из самых простых классов условий в проекте задача поиска NP-трудна [12].

Ранее проект использовался из командной строки, где каждый алгоритм запускался один раз за всё время жизни процесса. На его основе было сделано веб-приложение, которое сейчас не активно.

В 2022 году проект был оформлен в виде библиотеки на Python¹. Когда это случилось, все процессы работы с алгоритмами были разделены на две стадии: загрузка данных и исполнение. Это было сделано для внешнего сходства с `scikit-learn` [16], где работа с предоставляемыми алгоритмами ведётся через пару методов `fit` и `predict`.

Каждый алгоритм представлен в виде отдельного класса, наследующегося от основного класса `Algorithm`, предоставляющего функциональность, связанную с конфигурацией.

Алгоритмы библиотеки позволяют считывать данные из файлов CSV и из объектов `DataFrame` библиотеки `pandas` [19].

Для сборки с самого начала проекта использовалась система автоматизации сборки CMake [3]. Инструмент CTest, поставляемый с ней, используется сейчас при тестировании.

Ранее в представлении проекта на публику делался большой упор на использование закономерностей в рабочих процессах над данными как базовых элементов или “примитивов” (primitive) [5] и это слово излишне проникло в код — в какой-то момент даже базовый класс всех алгоритмов назывался `Primitive`². В наследованном коде проекта до сих пор остаются некоторые упоминания этого слова.

¹[PR 153](#) и [PR 160](#)

²<https://github.com/Desbordante/desbordante-core/pull/210>

Документация у проекта пока отсутствует из-за недостатка ресурсов и времени, но для обучения пользователей в репозитории присутствуют примеры в директории `examples`.

Эта работа описывает изменения в различных областях проекта Desbordante, от системы сборки и системы непрерывной интеграции (CI) до самого кода. Ниже будут описаны требуемые для понимания их части.

2.1. Запуск алгоритма

2.1.1. C++

Все алгоритмы наследуются от класса `Algorithm`, который предоставляет инструменты для задания конфигурации.

Запуск алгоритма в C++ происходит следующим образом:

1. Вызвать конструктор.
2. Задать значения параметров.
3. Вызвать метод `LoadData`.
4. Задать значения параметров.
5. Вызвать метод `Execute`.

Параметры задаются с помощью метода `SetOption`, значение параметра передаётся с помощью `boost::any`.

```
1 | class Algorithm {
2 |     // [...]
3 |     void SetOption(std::string_view option_name,
4 |                   boost::any const& value = {});
5 |     // [...]
6 | };
```

Работа с параметрами выполняется шаблонным классом `Option<T>`, в частности в нём происходит распаковка значения из `boost::any` и его непосредственная установка.

```

1 | template <typename T>
2 | std::vector<std::string_view> Option<T>::Set(
3 |         boost::any const& value) {
4 |     assert(!is_set_);
5 |     T converted_value = ConvertValue(value);
6 |     // ...
7 |     *value_ptr_ = std::move(converted_value);
8 |     // ...
9 | }
10 |
11 | template <typename T>
12 | T Option<T>::ConvertValue(
13 |         boost::any const& value) const {
14 |     // ...
15 |     if (value.type() != typeid(T))
16 |         throw ConfigurationError(/* ... */);
17 |     return boost::any_cast<T>(value);
18 |     // ...
19 | }

```

Для алгоритмов, использующих табличные данные, одной из опций, задаваемых перед вызовом `LoadData`, будет экземпляр класса, реализующего интерфейс для построчного их чтения, который называется `IDatasetStream`. Этот интерфейс реализуют классы для чтения данных из файла `CSV` и из `pandas.DataFrame`.

Методы `LoadData` и `Execute` реализуются классом `Algorithm`. Его потомками для определения существенного кода этих стадий определяются методы соответственно `LoadDataInternal` и `ExecuteInternal`.

2.1.2. Python

Предполагаемый интерфейс использования алгоритма в Python выглядит следующим образом:

```

1 | algo = desbordante.task.algorithms.Algorithm()

```

```

2 algo.load_data(param_l1=..., param_l2=..., ...)
3 algo.execute(param_e1=..., param_e2=..., ...)
4 for res in algo.get_patterns():
5     ...

```

Например,

```

1 algo = desbordante.md.algorithms.HyMD()
2 algo.load_data(
3     left_table=(ANIMALS_BEVERAGES, ',', True))
4 algo.execute(
5     column_matches=column_matches, min_support=11)

```

Здесь каждый параметр передаётся сначала в виде объекта Python, затем конвертируется с объект C++ и упаковывается в `boost::any`.

Для конвертации объектов алгоритм передаёт `std::type_index`, который является ключом в `std::unordered_map` в связывающем с Python коде³. По этому ключу располагается конвертирующая функция, производящая значение типа в C++ и оборачивающая его в `boost::any` для последующей передачи.

2.2. Система непрерывной интеграции

Desbordante имеет систему непрерывной интеграции, построенную на GitHub Actions.

Система включает в себя:

- Проверку форматирования с помощью clang-format;
- Статический анализ с помощью clang-tidy;
- Тестирование корректности работы алгоритмов;
- Проверку правильного вывода примеров использования;
- Регрессионное тестирование производительности;

³[src/python_bindings/py_util/py_to_any.cpp](#)

- Тестирование корректности работы из Python;
- Сборку wheel — формат бинарного распространения — для Python пакета.

Важные для этой работы детали:

1. инструмент clang-tidy для корректной работы требует создания `compile_commands.json` — файла, который содержит команды для компиляции, включающие в себя абсолютные пути до содержащих программные библиотеки директорий;
2. для реализации статического анализа в GitHub Actions используется clang-tidy-review [25];
3. тестирование корректности выполняется в 12 различных конфигурациях — четыре варианта сборки: Debug режим, Release режим, а также с санитайзерами UBSan (для нахождения неопределённого поведения) и ASan (для нахождения ошибок управления памятью) — и три компилятора: GCC, Clang, Apple Clang.

2.3. Зависимости

Desbordante использует в своей реализации семь библиотек: `emhash`, `ICU`, `Boost`, `Google Test`, `Magic Enum`, `pybind11`, `spdlog`. Далее следуют требуемые для понимания детали.

emhash `emhash`[27] предоставляет набор хеш-таблиц, оптимизированных под скорость работы в различных сценариях использования. Используется в одном из алгоритмов библиотеки.

ICU Библиотека ICU (International Components for Unicode) [10] предоставляется консорциумом Unicode для упрощения работы с этим стандартом в программах. Сейчас используется в проекте не слишком активно.

Boost Boost [2] является набором библиотек для различных задач. В проекте используются некоторые его части, например класс битовой маски, размер которой определяется во время выполнения (класс в C++ называется `dynamic_bitset`).

Google Test С помощью Google Test [9] выполняется тестирование корректности работы алгоритмов. С помощью средств этой библиотеки пишется код тестов, затем производится сборка тестов в исполняемый файл, а исполняемые файлы запускаются с помощью CTest — программы для тестирования, используемой в связке с CMake.

Magic Enum и Better Enums Magic Enum [13] позволяет получать данные о типах перечисления, недоступные в стандартном C++, например, список их членов или строку с названием одного из членов. Ранее в проекте использовалась библиотека Better Enums, однако она давно не обновлялась и предоставляла требуемую функциональность только для типов перечислений, созданных её средствами. Это усложняло работу с ней, поэтому она была заменена [29].

pybind11 Библиотека pybind11 [11] предоставляет средства для упрощения связывания кода на C++ с Python. Также с библиотекой поставляются команды для упрощения сборки модулей для Python с помощью CMake, которые устанавливают некоторые разумные стандартные опции. Модули для Python имеют вид файла разделяемой библиотеки.

spdlog Библиотека spdlog [15] предоставляет средства для протоколирования работы программы. Сообщения в ней отправляются в логгеры (logger), который перенаправляет их в стоки (sink). Логгеры можно получать по имени из реестра логгеров (registry) и регистрировать их в нём. Ранее для задач протоколирования в проекте использовалась библиотека Easylogging++ [26], однако она перестала обновляться, поэтому её пришлось заменить [30].

2.4. Сборка

Концептуально проект разделён на три части — основной код или “ядро” (`src/core`), код для связывания с Python или просто “связка с Python” (`src/python_bindings`) и тесты (`src/tests`).

Для автоматизации сборки проект использует систему CMake. Проекты в CMake оперируют с целями (`target`), каждая из которых является названием некоторого набора свойств [17]. Цели могут соответствовать, например, программной библиотеке или исполняемому файлу. Сборка задаётся командами в файлах `CMakeLists.txt`. CMake также позволяет задавать собственные команды.

Заголовочные файлы, в соответствии с Google Style Guide [8], включаются только по относительным путям от базовой директории `src/`.

2.4.1. Переработка кода CMake

На старте проекта сборка не была активно продумана, и просто модифицировалась по ходу развития. В PR 652⁴ система сборки была переработана с опорой на современные практики использования CMake [28].

Особенности системы сборки до PR 652:

- при сборке все файлы с кодом собирались неявно, путём поиска всех файлов, содержащих `.cpp` и `.h`⁵, для сборки нового кода отдельных действий не требовалось;
- ядру и связке с Python соответствовало по одной цели (программные библиотеки)^{6,7}, тестам — две: тестирование корректности и регрессионное тестирование производительности (исполняемые файлы)⁸;
- используемый стандарт C++ задавался для всего проекта⁹;

⁴<https://github.com/Desbordante/desbordante-core/pull/652>

⁵`src/core/CMakeLists.txt` (коммит `a8073e5fe812571846742e2352a28e9e16c853e7`, строка 7)

⁶`src/core/CMakeLists.txt` (коммит `a8073e5fe812571846742e2352a28e9e16c853e7`, строка 15)

⁷`src/python_bindings/CMakeLists.txt` (коммит `a8073e5fe812571846742e2352a28e9e16c853e7`)

⁸`src/tests/CMakeLists.txt` (коммит `a8073e5fe812571846742e2352a28e9e16c853e7`, строки 21 и 45)

⁹`CMakeLists.txt` (коммит `a8073e5fe812571846742e2352a28e9e16c853e7`, строка 105)

- объём кода на CMake был небольшим.

Из-за недочётов при оценке задачи, недостаточно чёткой постановки и иных обстоятельств переработку пришлось принимать несоответствующей стандартам индустриальной разработки.

Требованиями к новой системе сборки были:

- возможность отдельной сборки каждого алгоритма разработчиками — сборка всего проекта занимает много времени;
- явное указание компилируемых файлов — система сборки раньше могла ошибочно собирать файлы, которые не нужно было собирать, что могло вызывать неудачные сборки, а при удалении и добавлении файлов возникали аномалии;
- приведение кода на языке CMake в соответствие с общепринятыми практиками.

Особенности системы сборки после PR 652:

- каждый файл, который нужно собрать, указывается явно, сборка нового кода должна быть отражена в `CMakeLists.txt`;
- каждому алгоритму в ядре, каждому тесту и каждому элементу в части связки с Python соответствует собственная цель, определяемая с помощью функции в CMake;
- стандарт C++ и свойства задаются отдельно для каждой цели;
- количество кода на языке CMake выросло.

Команда для указания исходных файлов цели стала выглядеть следующим образом:

```
1 target_sources(  
2     ${NAME}  
3     PRIVATE [ SOURCE CODE FILES ]  
4     PUBLIC FILE_SET HEADERS BASE_DIRS  
5     "${PROJECT_SOURCE_DIR}/src"  
6 )
```

Две последние строки указывают базовую директорию для заголовочных файлов.

Стоит также обратить внимание, что заголовочные файлы перечислять в этой команде не обязательно.

При добавлении алгоритма используется определённая в CMake функция `desbordante_add_lib`¹⁰. Эта функция устанавливает для целей правильные свойства, в частности производится установка корректного стандарта C++ путём применения команды `target_link_libraries` с аргументом `desbordante.compile_feats`¹¹.

2.4.2. Связка с Python

Связка с Python использует библиотеку `pybind11` [11]. Эта библиотека компилируется с флагом `-fvisibility=hidden`, что означает, что к её символам не может быть получен доступ при компоновке, если соответствующие им части кода (например, классы или функции) не помечены явно. Из-за этого включение её объектов в объекты библиотек, которые собираются без этого флага компиляции, может вызывать ошибки, что обозначено в секции часто задаваемых вопросов в её документации [7].

Эти ошибки проявились в одной из версий новой системы сборки, и были устранены установкой флага на всех целях, файлы которых находились в `src/python_bindings`. Но после этого тестирование корректности в Python в системе CI обнаружило некорректную компоновку на MacOS.

Установка данного флага при сборке кода проекта была в планах ранее, поскольку правильная видимость символов имеет несколько положительных эффектов для собранной библиотеки [22]:

- ускорение её загрузки — то есть, `import desbordante` должно происходить быстрее;
- улучшение работы оптимизатора;

¹⁰[cmake/desbordante_helpers.cmake](#), строка 46

¹¹[cmake/desbordante_helpers.cmake](#), строка 8

- уменьшение размера файла библиотеки.

2.5. Поиск функциональных зависимостей

В ходе этой работы был изменён код всех алгоритмов в Desbordante, но существенные изменения, относящиеся непосредственно к их выполнению, были выполнены для трёх из них: НуFD [14], AID-FD [1], FD_Mine [24]. Все три этих алгоритма выполняют поиск функциональных зависимостей — это один из классов условий, алгоритмы поиска для которого поддерживаются в Desbordante. Функциональную зависимость можно определить следующим образом:

Определение 1 *Функциональная зависимость (далее FD) $X \rightarrow A$ является парой из набора атрибутов X и атрибута A некоторого отношения R , то есть $X \subseteq R, A \in R$. Мы говорим, что FD $X \rightarrow A$ соблюдается на экземпляре отношения r ($FdHolds[(X, A), r] \iff$*

$$\forall (p, q) \in (r \times r) (\forall B \in X \ p[B] = q[B] \implies p[A] = q[A])).$$

Называем X левой частью, A — правой частью.

Факт 1 *Очевидно, что для двух FD $X_1 \rightarrow A$ и $X_2 \rightarrow A$ верно*

$$X_1 \subseteq X_2 \implies (FdHolds[(X_1, A), r] \implies FdHolds[(X_2, A), r])$$

Используя этот факт, можно задать частичный порядок для множества FD с одной и той же правой частью по включению левых частей. Этот порядок используется для сокращения размера ответа в задаче поиска — множества соблюдающихся на некотором экземпляре отношения FD — вместо всего множества возвращаются только минимальные относительно этого порядка элементы (назовём это стандартной формой ответа). Также этот порядок задаёт структуру на множестве всех возможных функциональных зависимостей на данных, которой пользуются алгоритмы поиска.

Для некоторых пользовательских задач бывает полезным ограничить размер левых частей в ответе. В Desbordante это ограничение реализовано для всех алгоритмов поиска FD, однако не все из них используют его для сокращения времени поиска.

Для представления набора столбцов из таблицы алгоритмы Desbordante часто используют битовые маски. Одним из классов, который предоставляет интерфейсы для работы с ними, является `Vertical`.

Для использования этого класса в контейнерах, требующих хеширования, определяется своя хеш-функция, которая не совпадает со стандартной, предоставляемой `Boost.DynamicBitset`, причём присутствуют два варианта. Выбор производится на этапе компиляции с помощью определения макроса `SAFE_VERTICAL_HASHING`. Первый вариант в качестве хеша использует значение `boost::dynamic_bitset::to_ulong`. По задумке первый вариант более эффективен, но вызывает ошибки, если битовая маска слишком велика. Второй вариант (когда макрос определён) обрезает битовую маску до размера, при котором ошибка не возникает. При этом для класса `dynamic_bitset` определена собственная хеш-функция, которая по неизвестной причине не используется. При обычной сборке до проведения данной работы этот макрос не определялся, то есть использовался первый вариант, что является неожиданным поведением для пользователя.

2.6. Детали изменённых алгоритмов

Уточнение промежуточного ответа. Алгоритмы НуFD и AID-FD тем или иным образом используют следующую последовательность действий:

1. Имеем предполагающийся в данный момент ответ (набор FD).
2. Рассматриваем пару записей — определяем, значения каких атрибутов равны.
3. Корректируем ответ, удаляя FD, которые точно не должны быть в ответе при учёте этой пары (то есть те, которым она противоре-

чит), и добавляя непосредственно следующие за ними в соответствии с частичным порядком.

В итоге, после проверки всех рассматриваемых пар записей, получается ответ, не противоречащий ни одной из них.

Корректировка ответа в FD_Mine. Конечный результат выполнения алгоритма FD_Mine в оригинальном варианте [24] не соответствует стандартной форме, так как может содержать неминимальные элементы. Поэтому при интеграции его в Desbordante была добавлена процедура минимизации [32].

2.7. Зависимости включения

Зависимости включения рассматриваются как пример условия, для определения которого требуется несколько таблиц. Кратко приведём их определение (более точно оно представлено в [6]).

Определение 2 Пусть R, S — заголовки (наборы столбцов), r, s — таблицы с соответствующими заголовками. Пусть $\bar{R} \subseteq R, \bar{S} \subseteq S, |\bar{R}| = |\bar{S}|$. $\bar{R}$ **включается** в $\bar{S}$, если

$$\forall t_r \in r \quad \exists t_s \in s \quad t_r[\bar{R}] = t_s[\bar{S}].$$

В Desbordante такие условия отображаются в виде, представленном ниже.

(название таблицы 1, [названия столбцов таблицы 1])
-> (название таблицы 2, [названия столбцов таблицы 2])

Отображаемые названия таблиц получаются путём вызова методов соответствующих этим таблицам экземпляров классов, реализующих интерфейс `IDatasetStream`.

2.8. Случайные ошибки тестирования

В процессе тестирования в GitHub Actions иногда произвольно возникают ошибки, которые пропадают после перезапуска.

В каком-то случае это ошибки скачивания образов, используемых в тестировании, что в целом можно игнорировать. Однако до проведения описываемой в этом отчёте работы ошибки возникали в одних и тех же тестах на случайных конфигурациях с системой MacOS, и из-за некорректных настроек в системе CI подробная информация не выводилась.

Здесь также играет роль то, что конфигураций на одной системе проверяется сразу четыре, поэтому проявления ошибок вследствие, например, неопределённости параллелизма более вероятны.

2.9. Устаревший код

2.9.1. PositionListIndex

Многие алгоритмы концептуально используют структуры, называемые индексами списков позиций. С их помощью определяется, в каких записях (позициях) в таблице находится значение.

Такую структуру данных можно реализовать как массив массивов позиций, где индексом внешнего массива является идентификатор значения. При этом внутренние массивы длиной 1 можно исключить без потери информации [18].

Одной из её реализаций, применяемых в алгоритмах, появившихся в Desbordante ранее остальных, является класс `PositionListIndex`. Несмотря на концептуальную простоту этой структуры, данный класс содержит данные, которые используются не всеми алгоритмами, что затрудняет его понимание. В целях упрощения кода необходимо задокументировать его применение и рассмотреть возможность замены, однако прежде следует точно определить его функциональность, для чего целесообразно попытаться упростить реализацию и исключить неиспользуемые элементы.

2.9.2. Возврат времени исполнения

До проведённой работы интерфейс алгоритмов предполагал возврат времени исполнения в миллисекундах. Ранее это было нужно для изменения производительности без учёта преобразования данных во внутреннее представление (например, см. [31]). Сейчас это значение не используется, а в Python оно не было выведено вовсе.

Кроме того, если ранее исполнение алгоритма заключалось в вызове одного метода, то теперь их два (`LoadData` и `Execute`), что дополнительно усложняет вычисление названного значения.

Однако из-за наличия такого интерфейса и его соблюдения в прошлых алгоритмах всё равно приходится его реализовывать, что без необходимости усложняет код. Для упрощения реализации этого интерфейса в проект была добавлена функция `TimedInvoke`. Проверяющим код приходится напоминать про неё^{12,13,14,15}, что дополнительно тратит их время.

2.9.3. `util::Progress`

Поскольку выполнение алгоритмов может занимать много времени, для нужд веб-приложения были реализованы шкалы прогресса в алгоритмах. На момент начала работы это было реализовано с помощью класса `util::Progress`. Им предоставлялись методы для сообщения о текущем этапе исполнения алгоритма и мере его продвижения. Они представлены ниже.

```
1 class Progress {
2 public:
3     void AddProgress(double val) noexcept;
4     void SetProgress(double val) noexcept;
5     std::pair<uint8_t, double> GetProgress()
6         const noexcept;
```

¹²https://github.com/Desbordante/desbordante-core/pull/543#discussion_r2031146463

¹³https://github.com/Desbordante/desbordante-core/pull/537#discussion_r2031200483

¹⁴https://github.com/Desbordante/desbordante-core/pull/465#discussion_r1855222043

¹⁵https://github.com/Desbordante/desbordante-core/pull/389#discussion_r1623412300

```

7   void ToNextProgressPhase() noexcept;
8   std::vector<std::string_view> const&
9       GetPhaseNames() const noexcept {
10      return phase_names_;
11  }
12 };

```

Предполагалось, что каждый алгоритм имеет фиксированное количество этапов, через которые он продвигается последовательно, сообщая процент продвижения по ним во вне с помощью этого класса.

Возможность отображения прогресса не использовалась в Python и часто не реализовывалась алгоритмами, но, точно так же как и возврат времени, усложняла код — к примеру, промежуточным классам в иерархии наследования между классом конкретного алгоритма и базового `Algorithm` приходилось передавать набор стадий конструктору родительского класса.

В сочетании с тем, что веб-приложение, во-первых, уже довольно долго неактивно и, во-вторых, будет использовать модуль в Python, если разработка продолжится, класс `util::Progress` и связанные с ним вызовы стал ненужным.

При этом нужда в демонстрации прогресса исполнения алгоритма пользователям в Python ещё присутствует и возможность даже была запрошена [20]. Тем не менее, в имеющемся состоянии класс трудно интегрировать с Python. Однако информацию о стадиях, которая ему передавалась, было бы разумно сохранить.

2.9.4. Связывание с Python

Изначальный вариант связывания библиотеки с Python был реализован автором этой работы, когда он имел сильно меньший опыт написания программ на C и C++, за довольно небольшое время. С того момента тот код был реорганизован и частично переписан, однако в нём всё ещё довольно много проблемных мест.

Помимо этого, из-за допущения о том, что алгоритм будет запус-

каться только один раз за время жизни процесса, актуального только в начале проекта, не представлялось возможности корректно реализовать время жизни объектов условий, поскольку они хранили часть общего состояния внутри объектов алгоритмов. Попытка справиться с этим изначально заключалась в установке политики времени жизни для возвращённого значения (return value policy) в `pybind11`, однако позже обнаружилось, что это не было корректным решением. На данный момент этот дефект исправлен¹⁶ и от новых объектов условий требуется самостоятельное время жизни — в таком случае стандартная политика работает корректно.

Python модуль `desbordante` имеет определённую структуру. Модуль разделён на модули второго уровня, каждый из которых соответствует задаче (поиск или проверка соблюдения для некоторого типа закономерности) за некоторыми исключениями.

```
1 >>> import desbordante
2 >>> dir(desbordante)
3 [..., 'ac', 'afd', 'afd_metric_calculation',
4 'afd_verification', 'aind', 'aind_verification',
5 'aod_verification', 'ar', 'ar_verification',
6 'aucc_verification', 'cfd', 'cfd_verification',
7 'cind', 'cind_verification', ...]
```

Такие модули второго уровня, в свою очередь, содержат некоторые классы, связанные с типом закономерности, и модуль третьего уровня `algorithms`, в который включаются алгоритмы для выполнения задачи. Класс `Default` в `algorithms` обозначает алгоритм, который предлагается использовать в отсутствие иной информации — обычно это самый современный алгоритм.

Для упрощения задания такой типовой структуры в коде для связывания с Python присутствует функция `BindPrimitive`, которая, в частности, принимает аргумент политики времени жизни для возвращённого значения. Аргумент уже не актуален и может ввести в заблуждение

¹⁶<https://github.com/Desbordante/desbordante-core/pull/496>

людей, создав неправильное представление, что установка нестандартной политики в принципе допустима.

2.10. Недостающие возможности

2.10.1. Вспомогательные интерпретаторы

В версии Python 3.12 была добавлена поддержка вспомогательных интерпретаторов (sub-interpreter) [23]. Это были первые варианты улучшенной поддержки поддержки параллельности в языке.

pybind11 стандартно не разрешал использование модулей с его использованием во вспомогательных интерпретаторах, поскольку многие расширения не были рассчитаны на такое использование. Для Desbordante это тоже было верно из-за неуместного использования глобальных переменных.

2.10.2. Различение DataFrame

Некоторые закономерности, представленные в Desbordante, заданы на нескольких таблицах. При их выводе для человека полезно различать, к какой таблице относится, например, столбец.

Однако, поскольку у DataFrame в pandas отсутствует стандартный атрибут с названием таблицы, код в Desbordante при получении даёт ему стандартное название “Pandas DataFrame”¹⁷, из-за чего возникают проблемы с читаемостью результата [21].

¹⁷[src/python_bindings/py_util/create_dataframe_reader.h](#)

3. Проведённая работа

3.1. Процесс сборки

3.1.1. Видимость символов

Вследствие более подробного рассмотрения протоколов тестирования на MacOS были обнаружены ошибки при преобразовании переданного значения. Материалы gcc wiki про опцию видимости [22] упоминали несоответствия объектов `std::type_info`, поэтому была установлена стандартная видимость для всех типов, используемых как параметры, что устранило ошибки.

Текущее объяснение подобного поведения является следующим — типы, используемые в качестве параметров, включаются в заголовочные файлы с алгоритмами и при сборке с флагом `-fvisibility=hidden`, определяются со скрытой видимостью, а при обычной сборке — со стандартной. Компоновщик на MacOS считает их различными символами, а компоновщик, используемый при сборке на Linux, сливает в один. Последнее прямо указывается на gcc wiki [22]:

```
typeinfo symbols for classes without a vtable are defined on demand within each object file that uses the class for EH and are defined weakly so the definitions get merged at link time into one copy.
```

Основной причиной, по которой проект не использовал эту опцию ранее были как раз подобные проблемы с компоновкой. Но так как в текущем состоянии всё равно приходится с ними сталкиваться, она стала более не актуальной. Поэтому в PR 693¹⁸ была установлена скрытая видимость символов. В PR 787¹⁹ были исправлены ошибки, связанные с переходом с Better Enums на Magic Enum — типы перечисления, определяемые средствами первой, были заменены на обычные типы перечисления, но настройка видимости при этом преобразовании была потеряна.

¹⁸<https://github.com/Desbordante/desbordante-core/pull/693>

¹⁹<https://github.com/Desbordante/desbordante-core/pull/787>

3.1.2. Хеширование битовых масок

При профилировании кода было обнаружено, что хеширование для `Vertical` и функции, связанные с ним, практически не используются (менее 0.01% времени для всех из них), а значит не могут существенно влиять на производительность. Поэтому в интересах поддерживаемости кода функция хеширования для этого класса была заменена на стандартную в PR 788²⁰.

Соответственно, была удалена и опция системы сборки, с помощью которой выбирался способ хеширования.

3.1.3. Исправление дефектов новой системы сборки

Повторяющийся код. Поскольку для всех целей в Desbordante базовая директория для заголовочных файлов одна (`src/`) в PR 698²¹ она стала устанавливаться на этапе создания цели.

Не включённые части. Файлы некоторых целей не собирались вообще. Это проблема, которая заведомо не могла возникнуть в прошлой системе сборки, так как там собирались все файлы. Было исправлено несколько таких ошибок в PR 698²² и PR 769²³.

Стандарт C++ для регрессионного тестирования производительности. Было обнаружено, что для кода регрессионного тестирования производительности не был установлен стандарт C++, что было исправлено в PR 773²⁴. Так как стандартной опцией в gcc и так является стандарт C++20, это не изменило сборку, но изменение позволит предотвратить неожиданные проблемы со сборкой, когда в соответствующем коде будут использоваться возможности C++23.

²⁰<https://github.com/Desbordante/desbordante-core/pull/788>

²¹<https://github.com/Desbordante/desbordante-core/pull/698>

²²<https://github.com/Desbordante/desbordante-core/pull/698>

²³<https://github.com/Desbordante/desbordante-core/pull/769>

²⁴<https://github.com/Desbordante/desbordante-core/pull/773>

3.2. Автоматические проверки кода

3.2.1. clang-tidy-review

После одного из обновлений clang-tidy-review стало использовать версию Ubuntu, отличную от используемой при сборке²⁵.

Из-за использования ими разных версий Python сгенерированный на одной из версий файл `compile_commands.json` будет содержать пути, которые будут некорректными для другой. Для того, чтобы эта проблема больше не возникала, его создание было перенесено в сам контейнер в PR 721²⁶.

3.2.2. Google Test

В PR 708²⁷ к опциям CTest в системе CI был добавлен флаг для вывода результатов запусков с ошибками `--output-on-failure`.

Ранее вывод при ошибках во время тестирования выглядел следующим образом:

```
Start 973: StaticMapTest.FindAndAtWork
1/1 Test #973: StaticMapTest.FindAndAtWork .....***Failed
0.00 sec
...
The following tests FAILED:
 973 - StaticMapTest.FindAndAtWork (Failed)
Errors while running CTest
Output from these tests are in:
.../build/Testing/Temporary/LastTest.log
Use "--rerun-failed --output-on-failure" to re-run the failed
cases verbosely.
```

Вывод тестов оказывался в файловой системе вместо стандартного вывода, который отображается в протоколе запуска в GitHub Actions.

²⁵<https://github.com/ZedThree/clang-tidy-review/commit/8918b9f83a7779809ffd7f404c84c3c1b4231af3#diff-dd2c0eb6ea5cfc6c4bd4eac30934e2d5746747af48fef6da689e85b752f39557R1>

²⁶<https://github.com/Desbordante/desbordante-core/pull/721>

²⁷<https://github.com/Desbordante/desbordante-core/pull/708>

Теперь вывод при ошибках во время тестирования выглядит так:

```
Start 973: StaticMapTest.FindAndAtWork
1/1 Test #973: StaticMapTest.FindAndAtWork .....***Failed
0.00 sec
...
[ RUN      ] StaticMapTest.FindAndAtWork
.../src/tests/unit/test_static_map.cpp:23: Failure
Expected equality of these values:
    *value
      Which is: 20
    201

[ FAILED   ] StaticMapTest.FindAndAtWork (0 ms)
...
The following tests FAILED:
  973 - StaticMapTest.FindAndAtWork (Failed)
Errors while running CTest
```

В отличие от прошлого случая, ошибка отображается в стандартном выводе.

3.2.3. Инициализация логгирования

После добавления флага для CTest была обнаружена ошибка, связанная с инициализацией логгера Desbordante. После рассмотрения кода, связанного с ним, был выявлен программный дефект — логгер создавался независимо в каждом потоке, и была возможна ситуация, где один и тот же логгер добавлялся в реестр, что вызывало ошибку.

Эта ошибка была исправлена в PR 709²⁸, инициализация логгирования была упрощена. Добавление в реестр было вовсе удалено, поскольку функциональность, для которой оно требуется, не используется.

²⁸<https://github.com/Desbordante/desbordante-core/pull/709>

3.3. Неиспользуемая функциональность

3.3.1. `util::Progress`

Из-за сложности интеграции с Python и заброшенным состоянием функциональности часть с учётом прогресса алгоритмов была удалена в PR 686²⁹.

Значимая реализация учёта прогресса присутствовала только в 8 алгоритмах, только в трёх из них были указаны названия стадий:

- Depminer
 1. AgreeSets generation.
 2. Finding CMAXSets.
 3. Finding LHS.
- FastFDs
 1. Agree sets generation.
 2. Finding minimal covers.
- Cords
 1. Calculating values frequencies.
 2. SFD and correlations mining.

3.3.2. Возврат времени исполнения и иной учёт времени

Из-за неактуальности возврата времени исполнения эта часть интерфейса алгоритмов была удалена в PR 744³⁰ и PR 725³¹.

Помимо этого, во время удаления были обнаружены ещё несколько зон в проекте, которые без надобности учитывали время. Наиболее заметная из них — вывод времени исполнения некоторых методов в

²⁹<https://github.com/Desbordante/desbordante-core/pull/686>

³⁰<https://github.com/Desbordante/desbordante-core/pull/744>

³¹<https://github.com/Desbordante/desbordante-core/pull/725>

протокол. Протоколирование этого показателя только очень опосредованно может помочь в поисках ошибок разработчикам, а пользователю такие замеры времени не нужны, то есть в большинстве случаев такой вывод добавляет ненужную информацию в протокол. Если эти замеры нужны самим разработчикам алгоритма для профилирования, то они могут сами модифицировать код. При этом этот вывод затрудняет чтение кода. Поэтому было принято решение также удалить весь вывод времени исполнения в протокол.

После этого также была удалена функция `TimedInvoke`, поскольку для неё больше не было применения. Многие места, где она вызывалась, являлись методами выполнения алгоритма `ExecuteInternal`, в которых выполнялся возврат её значения, а метод, непосредственно содержащий код, реализующий алгоритм, вызывался опосредованно через неё.

```
1 unsigned long long GddValidator::ExecuteInternal() {  
2     return util::TimedInvoke(  
3         &GddValidator::FilterValidGdds, this);  
4 }
```

В таких местах вызовы `TimedInvoke` были заменены на код реализующего алгоритм метода, что упростило понимание.

3.3.3. Время жизни в Python

Из функции `BindPrimitive` параметр политики времени жизни объектов закономерностей в PR 689³² был убран, поскольку они должны иметь самостоятельное время жизни, в связи с чем стандартная политика должна работать всегда.

3.3.4. `PositionListIndex`

Из класса `PositionListIndex` в PR 713³³ был удалён весь код, связанный со значениями `NULL`, поскольку он был некорректен — в другой

³²<https://github.com/Desbordante/desbordante-core/pull/689>

³³<https://github.com/Desbordante/desbordante-core/pull/713>

части проекта пустая строка не является значением `NULL`, в отличие от этого кода.

Также в PR 717³⁴ из него было удалено поле `original_relation_size_`.

3.4. Повторный обзор кода

Алгоритмы `HyFD`, `FD_Mine`, `AID-FD` были рассмотрены повторно.

Способ получения ответа в `AID-FD` и `HyFD` позволяет ускорить их при указанной опции `max_lhs` — в рамках корректировки ответа, описанной в разделе 2.6, при превышении размера левой части можно не добавлять зависимость и ответ останется корректным. Это было реализовано в PR 785³⁵ и PR 714³⁶.

В `AID-FD` одна из процедур была улучшена — добавлен `std::move` и более не используется метод `at` в словаре, что позволяет компилятору оптимизировать лучше³⁷.

Стадия корректировки ответа в `FD_Mine` также была улучшена — ранее там использовался `std::unordered_map` с типом значения `bool`, которое всегда устанавливалось в `true`, и неоптимальные способы доступа, из-за которых вычисление ячейки в структуре происходило чаще, чем необходимо. Коллекция была заменена на `std::unordered_set` и использование было исправлено. Также был добавлен отсутствующий вызов `std::move`³⁸.

В PR 715³⁹ были устранены схожие небольшие ошибки — добавлен вызов `std::move` и устранено неоптимальное использование битовых масок. Для операции исключения установленных битов использовалась последовательность операций `a &= ~b`, которая выделяет память для инвертированной маски `~b`. Она была заменена на предоставляемую Boost операцию `a -= b`.

³⁴<https://github.com/Desbordante/desbordante-core/pull/717>

³⁵<https://github.com/Desbordante/desbordante-core/pull/785>

³⁶<https://github.com/Desbordante/desbordante-core/pull/714>

³⁷[КОММИТ 8fb8fafce57371ab0eafac4bcc47a8a0bbc080fa](#)

³⁸[КОММИТ 525d6ebdf4e5b658df8c4fb0ba064656920c7fcd](#)

³⁹<https://github.com/Desbordante/desbordante-core/pull/715>

3.5. Добавление недостающих возможностей

3.5.1. Поддержка множественных интерпретаторов

Библиотека Desbordante интуитивно должна без проблем поддерживать множественные интерпретаторы — алгоритмы берут данные и преобразуют их в ответ, фоновые потоки отсутствуют, глобального изменяемого состояния не должно быть. Но при просмотре оно было обнаружено.

В PR 729⁴⁰ было удалено практически всё глобальное состояние. В PR 637⁴¹ включена поддержка множественных интерпретаторов.

В основном были обнаружены переменные, которые нужно было просто сделать постоянными, но несколько переменных были удалены или перенесены. Среди глобальных изменяемых переменных остались только переменные, связанные с логгированием, и один счётчик.

Глобальные переменные были обнаружены с помощью `clang-query` и следующей спецификации для сопоставления (`matcher`):

```
1 match varDecl(  
2   hasGlobalStorage(),  
3   isExpansionInMainFile(),  
4   unless(isConstexpr()),  
5   unless(hasType(isConstQualified())) ,  
6   unless(  
7     anyOf(  
8       isExpandedFromMacro("TEST_P"),  
9       isExpandedFromMacro("INstantiate_TEST_SUITE_P"),  
10      isExpandedFromMacro("TYPED_TEST_SUITE_P"),  
11      isExpandedFromMacro("TYPED_TEST_P"),  
12      isExpandedFromMacro(  
13        "INstantiate_TYPED_TEST_SUITE_P"),  
14      isExpandedFromMacro("TYPED_TEST"),  
15      isExpandedFromMacro("PYBIND11_MODULE")
```

⁴⁰<https://github.com/Desbordante/desbordante-core/pull/792>

⁴¹<https://github.com/Desbordante/desbordante-core/pull/637>

```
16 |         )
17 |     )
18 | )
```

Здесь выбираются все переменные с глобальным хранением, не из включённых заголовочных файлов, не являющиеся константными и не относящиеся к используемым макросам Google Test и pybind11.

3.5.2. Различение DataFrame

Поскольку `pandas.DataFrame` не имеют названий, название нужно передавать вместе с ним. Для подобных задач можно использовать следующие способы:

- создание собственного подкласса `DataFrame`⁴²;
- передавать название в кортеже с объектом;
- использование атрибута `attrs`⁴³.

Вариант с созданием собственного варианта `DataFrame` чрезмерно сложный, а предлагаемые средства рассчитаны на использование в Python. На данный момент модуль `desbordante` для Python реализуется исключительно из C++, что затруднит реализацию.

Реализация, в которой название передаётся в кортеже с объектом (`algo.load_data(table=(df, "name"))`) является приемлемым вариантом — достаточно изменить функцию преобразования в коде связки с Python для типа `IDatasetStream` — но был реализован не он.

Вместо этого в PR 759⁴⁴ был реализован вариант передачи названия с помощью `attrs`. Этот атрибут предназначен специально для передачи метаданных⁴⁵. Кроме того, реализация такой передачи названия наиболее простая — достаточно прочитать значение атрибута⁴⁶.

⁴²[Subclassing pandas data structures](#)

⁴³<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.attrs.html>

⁴⁴<https://github.com/Desbordante/desbordante-core/pull/759>

⁴⁵<https://github.com/pandas-dev/pandas/issues/2485#issuecomment-608227532>

⁴⁶[КОММИТ 7a4108d92803557442e49f1106a889d6874e49cd](#)

Для документирования данной возможности также был добавлен пример использования⁴⁷.

Однако прочтение атрибута не поможет различить таблицы, если этот атрибут не задан. Для этого случая в PR 789⁴⁸ было реализовано добавление индекса в стандартное название таблицы.

⁴⁷[коммит f838f0c8a74c036cabe4191aa1e8bdd6e108fb20](#)

⁴⁸<https://github.com/Desbordante/desbordante-core/pull/789>

4. Анализ

4.1. Проверка эффектов скрытых символов

На вики-сайте GCC преимуществами использования скрытой видимости символов названы уменьшение размера файла библиотеки, ускорение загрузки разделяемой библиотеки и улучшение возможностей оптимизации [22]. Проверим эти эффекты.

Сравнивались модули Python по состоянию на коммит `babe3cb9785`⁴⁹ с применением скрытой видимости ко всем целям и только к целям, относящимся к связке с Python. Последнее было реализовано путём комментирования соответствующих строк в основном CML и добавления опций сокращения символов к целям в `src/python_bindings`. Флаги на Python-целях являются требованием применения `pybind11`.

Тестирование выполнялось на ноутбуке ASUS TUF Gaming F16 модели FX608JMR, Intel Core i7-14650HX. Кулеры ЦП и видеокарты были установлены на максимальную скорость (6600 оборотов в минуту). С помощью `x86_energy_perf_policy` были установлены `EBP = 0` (режим максимальной производительности), `HWP` максимально возможное для всех ядер, режим Turbo был отключен для повышения стабильности тестирования.

Размер Python-модуля со стандартной видимостью символов — 17.5 МБ, со скрытой — 16.4 МБ. Стоит отметить, что функции CMake, поставляемые с `pybind11`, применяют команду `strip` к получаемым файлам, что повлияло на размер в обоих случаях.

Выполнить сравнение скорости загрузки модуля в Python не представляется возможным, поскольку она занимает всего около 0.12 секунд, что находится слишком близко к уровню погрешности — время выполнения `python -c ''` — около 0.01 секунды, время выполнения `python -c 'import this'` (простой вывод текста) — около 0.02 секунды.

Был также выполнен тест на время выполнения алгоритма НуFD

⁴⁹<https://github.com/Desbordante/desbordante-core/tree/babe3cb9785ad5f1d617a1d271982a7c78d7dd52>

на наборе данных iowa1kk. Тест выполнялся после перезагрузки 12 раз подряд, с перерывами в 30 секунд. В случае со скрытыми символами средним временем выполнения было 38.66 секунды, с выборочной дисперсией 0.62. Для стандартной видимости соответствующими значениями были 38.45 и 0.63. Р-значением t-статистики для таких результатов оказалось 0.55, что значит, что средние, скорее всего, равны. То есть влияние сокрытия символов на производительность в рамках данного теста статистически незначимо.

Итого, в этом проекте применение скрытой видимости символов имело заметный эффект только на размере библиотеки.

4.2. Уменьшение количества кода

Некоторые из изменений были заявлены как сокращающие количество кода в проекте. Они приведены в таблице 1. В первом столбце указано изменение и ссылка на его описание, во втором — количество удалённых строк, в третьем — комментарий об изменениях. Второй столбец был получен с помощью `git diff --shortstat`, сравнивались коммиты перед принятием соответствующего PR и после, количество получилось вычитанием количеств удалённых строк из добавленных, для строки с учётом времени были сложены два составляющих PR.

На момент коммита [b65df51e5a1e7e42b48d994a86cffe65c239f98](#)⁵⁰ в файлах кода проекта присутствует около 90000 строк:

```
1 $ git ls-files \  
2   | grep '\(\\.\\(h\\|cpp\\)\\)\\|CMakeLists.txt' \  
3   | xargs cat | wc -l  
4 90742
```

Замечание об указании стандарта. Стандарт для всех целей можно указать с помощью переменной `CMake CMAKE_CXX_STANDARD`, что сократит количество кода и полностью устранил ошибки как в разделе 3.1.3. Однако это не позволит распространить требование на це-

⁵⁰[b65df51e5a1e7e42b48d994a86cffe65c239f98](#)

Таблица 1: Уменьшение количества кода в проекте

изменение	стр.	комментарий
CustomHashing (3.1.2)	45	удалена опция проекта CMake, удалён код
BASE_DIRS (3.1.3)	262	в основном удалён CMake код
удаление util::Progress (3.3.1)	237	функциональность в неактуальном состоянии
учёт времени (3.3.2)	685	неактуальная функциональность, 122 файла изменено, затронуты все алгоритмы
RV policy (3.3.3)	8	лишний параметр
PLI null (3.3.4)	102	позволило удалить параметр во многих алгоритмах
original_relation_size_ (3.3.4)	8	лишнее поле

ли CMake, соединяющиеся с целями проекта с помощью, например, `target_link_libraries`. Так как ядро Desbordante само является библиотекой, это недопустимо. Поэтому на данном этапе приходится только принимать возможность подобных ошибок, что вполне приемлемо, так как новые части в проект добавляются нечасто. В целом, можно создать процесс добавления новых частей, который исключит возможность такой ошибки, однако по той же причине это сейчас не в приоритете.

4.3. Система непрерывной интеграции

Сборка всех частей. `compile_commands.json` теперь генерируется в ходе действия `clang-tidy-review`. Однако после его переноса его пришлось редактировать, чтобы добавить проверку кода регрессионного тестирования производительности⁵¹. Это обнаружило проблему — добавление новых частей проекта, отражённое в системе сборки, должно быть синхронизировано с GitHub Actions. Сейчас правильным решением представляется добавление опции системы сборки, которая включа-

⁵¹<https://github.com/Desbordante/desbordante-core/pull/746>

ет сборку всех частей проекта.

Проверка корректной видимости. Для избежания ошибок, связанных с видимостью, следует сделать соответствующие проверки в GitHub Actions. Автором не было подробно исследованы возможности такой проверки, однако на данный момент представляется возможность использования инструментов `readelf`, `nm` и `c++filt` для обнаружения символов с неправильной видимостью. Например, такими являются символы с видимостью “hidden”, соответствующие `typeinfo` для типов, используемых как параметр шаблона `Option<T>`.

Проверка включения в систему сборки. Текущие проверки в системе CI никак не могут обнаружить файлы, которые не собираются. Устранить это можно, используя средства CMake, собрав включённые в цели файлы и сравнив со списком всех файлов, что и стоит сделать. Кроме того, стоит использовать функциональность `FILE_SET` для явной привязки заголовочных файлов к целям, что позволит производить такую проверку и для них. Также это улучшит интеграцию с IDE, которые смогут отображать заголовочные файлы как часть целей.

Найденные программные дефекты. После включения вывода тестирования при неудаче, автору удалось найти дефект в инициализации логгирования в PR 709⁵². Также вследствие данного изменения была обнаружена и в PR 753⁵³ исправлена ошибка в деструкторе одного из классов, которая проявлялась только на MacOS. То есть это изменение было исключительно полезным и с его помощью в итоге удалось исключить случайные неудачи в системе CI по зависящим от проекта причинам.

Глобальные переменные. В связи с поддержкой вспомогательных интерпретаторов глобальные переменные нужно запретить с помощью

⁵²<https://github.com/Desbordante/desbordante-core/pull/709>

⁵³<https://github.com/Desbordante/desbordante-core/pull/753>

проверок в системе CI, кроме особенных случаев. Спецификация сопоставления, использовавшаяся для их поиска, была передана в clang-query как параметр вручную, но её также можно использовать для проверки с помощью clang-tidy.

Инструментирование кода. В системе CI сейчас используются только санитайзеры UBSan, ASan. Следует добавить и проверки с помощью других инструментов наподобие valgrind, чтобы уменьшить вероятность ошибок наподобие описанной в секции 3.2.3. Однако это нужно балансировать с доступным ресурсом машин в GitHub Actions — больше тестов означает больше времени на прохождение тестирования.

4.4. Возможности Desbordante

4.4.1. Различение DataFrame

Небольшое изменение, описанное в секции 3.5.2, сильно улучшает пользовательский опыт. Ранее зависимость включения (inclusion dependency) при чтении из DataFrame не отображала названий таблиц, что затрудняло читаемость.

```
(Pandas dataframe, [Department name])  
-> (Pandas dataframe, [Department name])
```

Теперь в этом случае зависимость включения как минимум отображает индексы таблиц, переданных в параметрах.

```
(Pandas dataframe (3), [Department name])  
-> (Pandas dataframe (1), [Department name])
```

При использовании `attrs["name"]` есть возможность увеличить читаемость ещё больше.

```
(student, [Department name]) -> (department, [Department name])
```

4.4.2. Отслеживание прогресса

Поскольку времена исполнения алгоритмов могут исчисляться часами, днями или даже неделями из-за сложности решаемых задач, было бы полезно иметь некоторый механизм обратной связи по тому, как алгоритм выполняется.

Однако его реализация в виде “процента выполнения” часто может быть довольно затруднительной. Если алгоритм заключается, например, в проверке всех возможных пар записей, то можно просто подсчитывать количество проверенных на данный момент пар. Но если алгоритм начинает использовать эвристики для ранней остановки и смены подхода, то предсказать его поведение становится значительно более сложно.

Тем не менее, полезными будут некоторые индикаторы того, что происходит “внутри”, чтобы пользователь знал, что алгоритм не завис. Поэтому в будущем разумнее будет реализовать не индикатор прогресса, а некоторый стандарт для сообщения алгоритмом его важного состояния.

4.4.3. Проверки производительности

В процессе работы с алгоритмами в Desbordante нередко получают результаты, достойные публикации на конференциях. Почти всегда такие публикации включают замеры производительности.

Но для пользователя алгоритма, которому нужен прежде всего результат, точное время, за которое он был получен, не важно. Важно только получить его достаточно быстро. Но, как было обнаружено при работе, описанной в разделе 3.3.2, много кода, измеряющего время исполнения, проникло в ядро проекта.

Следует закрепить в документации проекта требование не включать код для замеров производительности в алгоритмы и, возможно, использовать некоторые эвристики для обеспечения его соблюдения в системе CI.

Однако для библиотеки алгоритмов возможно и иное применение —

отслеживание их работы в целях улучшения. В таком случае некоторый код для этого был бы полезен. Вероятно, это можно реализовать с помощью одного механизма с выдачей обратной связи пользователю.

Заключение

В ходе работы были произведены различные улучшения кода проекта Desbordante.

1. Был повторно рассмотрен существующий код.
2. Было устранено несколько программных ошибок.
3. Существующий код проекта был упрощён.
4. Было добавлено две недостающие возможности.
5. Был проведён анализ сделанной работы.

Список литературы

- [1] [Approximate Discovery of Functional Dependencies for Large Datasets](#) / Tobias Bleifuß, Susanne Bülow, Johannes Frohnhofen et al. // Proceedings of the 25th ACM International on Conference on Information and Knowledge Management. — CIKM '16. — New York, NY, USA : Association for Computing Machinery, 2016. — P. 1803–1812. — URL: <https://doi.org/10.1145/2983323.2983781>.
- [2] Boost. — URL: <https://www.boost.org/> (дата обращения: 2026-06-27).
- [3] CMake - Upgrade Your Software Build System. — URL: <https://cmake.org/> (дата обращения: 2026-06-27).
- [4] [Desbordante: a Framework for Exploring Limits of Dependency Discovery Algorithms](#) / Maxim Strutovskiy, Nikita Bobrov, Kirill Smirnov, George Chernishev // 2021 29th Conference of Open Innovations Association (FRUCT). — 2021. — P. 344–354.
- [5] [Desbordante: from benchmarking suite to high-performance science-intensive data profiler](#) / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: <https://doi.org/10.1145/3703323.3703725>.
- [6] Fast Approximate Discovery of Inclusion Dependencies / Sebastian Kruse, Thorsten Papenbrock, Christian Dullweber et al. // Datenbanksysteme für Business, Technologie und Web (BTW 2017), 17. Fachtagung des GI-Fachbereichs „Datenbanken und Informationssysteme“ (DBIS), 6.-10. März 2017, Stuttgart, Germany, Proceedings / Ed. by Bernhard Mitschang, Daniela Nicklas, Frank Leymann et al. — Vol. P-265 of LNI. — GI, 2017. — P. 207–226. — URL: <https://dl.gi.de/handle/20.500.12116/629>.

- [7] Frequently asked questions - pybind11 documentation. — URL: <https://pybind11.readthedocs.io/en/stable/faq.html#someclass-declared-with-greater-visibility-than-the-type-of-its> (дата обращения: 2026-06-24).
- [8] Google C++ Style Guide. — URL: <https://google.github.io/styleguide/cppguide.html> (дата обращения: 2026-07-01).
- [9] GoogleTest User's Guide. — URL: <https://google.github.io/gtest/> (дата обращения: 2026-06-27).
- [10] ICU - International Components for Unicode. — URL: <https://icu.unicode.org/> (дата обращения: 2026-06-27).
- [11] Jakob Wenzel, Rhinelander Jason, Moldovan Dean. pybind11 – Seamless operability between C++11 and Python. — 2017. — <https://github.com/pybind/pybind11>.
- [12] Mannila Heikki, Räihä Kari-Jouko. On the complexity of inferring functional dependencies // [Discrete Applied Mathematics](#). — 1992. — Vol. 40, no. 2. — P. 237–243. — URL: <https://www.sciencedirect.com/science/article/pii/0166218X92900315>.
- [13] Neargye/magic_enum: Static reflection for enums (to string, from string, iteration) for modern C++, work with any enum type without any macro or boilerplate code. — URL: https://github.com/Neargye/magic_enum (дата обращения: 2026-06-27).
- [14] Papenbrock Thorsten, Naumann Felix. [A Hybrid Approach to Functional Dependency Discovery](#) // Proceedings of the 2016 International Conference on Management of Data. — SIGMOD '16. — New York, NY, USA : Association for Computing Machinery, 2016. — P. 821–833. — URL: <https://doi.org/10.1145/2882903.2915203>.
- [15] SPDLog – Blazing Fast C++ Logging Library with Async Support. — URL: <https://spdlog.org/> (дата обращения: 2026-06-27).

- [16] Scikit-learn: Machine Learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort et al. // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830.
- [17] Step 1: Getting Started with CMake. — URL: <https://cmake.org/cmake/help/latest/guide/tutorial/Getting%20Started%20with%20CMake.html#exercise-1-building-an-executable> (дата обращения: 2026-06-24).
- [18] Tane: An Efficient Algorithm for Discovering Functional and Approximate Dependencies / Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, Hannu Toivonen // [The Computer Journal](#). — 1999. — Vol. 42, no. 2. — P. 100–111.
- [19] The pandas development team. pandas-dev/pandas: Pandas. — 2026. — May. — URL: <https://doi.org/10.5281/zenodo.20127038>.
- [20] Using data frames with inclusion mining. — URL: <https://groups.google.com/g/desbordante/c/BqpE5dK0EXo/m/JYu2dxz6DgAJ> (дата обращения: 2026-06-27).
- [21] Using data frames with inclusion mining. — URL: <https://groups.google.com/g/desbordante/c/BqpE5dK0EXo/m/fZPhHKnUAwAJ> (дата обращения: 2026-06-27).
- [22] Visibility - GCC Wiki. — URL: <https://gcc.gnu.org/wiki/Visibility> (дата обращения: 2026-06-24).
- [23] What's New In Python 3.12. — URL: <https://docs.python.org/3.12/whatsnew/3.12.html#pep-684-a-per-interpreter-gil> (дата обращения: 2026-06-27).
- [24] Yao Hong, Hamilton Howard J. Mining functional dependencies from data // Data Min. Knowl. Discov. — 2008. — Apr.. — Vol. 16, no. 2. — P. 197–219.

- [25] ZedThree/clang-tidy-review: Create a pull request review based on clang-tidy warnings. — URL: <https://github.com/ZedThree/clang-tidy-review/> (дата обращения: 2026-07-01).
- [26] abumq/easyloggingpp: C++ logging library. It is powerful, supports asynchronous low latency, extendable, light-weight, fast performing, thread and type safe and consists of many built-in features. It provides ability to write logs in your own customized format. It also provide support for logging your classes, third-party libraries, STL and third-party containers etc. — URL: <https://github.com/abumq/easyloggingpp> (дата обращения: 2026-06-27).
- [27] emhash | Fast and memory efficient c++ flat hash table/map/set. — URL: <https://ktprime.github.io/emhash/> (дата обращения: 2026-06-27).
- [28] Гаврилов Артём. Модернизация CMake в проекте Desbordante. — URL: <https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/CMake%20modernization%20-%20Gavrilov%20Artyom%20-%202025%20autumn.pdf> (дата обращения: 2026-07-03).
- [29] Гараев Тагир. Desbordante: улучшение инфраструктуры. — URL: <https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/Desbordante%20infrastructure%20-%20Garaev%20Tagir%202025%20autumn.pdf> (дата обращения: 2026-07-03).
- [30] Гараев Тагир. Улучшение инфраструктуры журанализирования в Desbordante. — URL: <https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/Logging%20infrastructure%20-%20Garaev%20Tagir%20-%202025%20spring.pdf> (дата обращения: 2026-07-03).
- [31] Струтовский Максим. Реализация современных алгоритмов для поиска зависимостей в базах данных. — URL: <https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/>

[Pyro%20-%20Maxim%20Strutovsky%20-%202020%20spring.pdf](#)

(дата обращения: 2026-06-27).

- [32] Шукин Илья. Реализация алгоритма поиска функциональных зависимостей FD_Mine. — URL: https://github.com/Desbordante/desbordante-core/blob/f4f4ff997d00cfb55b52bdd58b9af84989e2fd1/docs/papers/FD_Mine%20-%20Ilya%20Shchuckin%20-%202021%20spring.pdf (дата обращения: 2026-06-27).